

```
/*
```

```
=====
```

Oracle Views – Professional Telecom Industry Use Cases

One-file lab script with comments, sample data, views, materialized view, and testing queries.

Tested conceptually for Oracle Database / SQL Developer.

Notes:

1. Run this script in a training/schema user with CREATE TABLE, CREATE VIEW, CREATE MATERIALIZED VIEW privileges.
2. GRANT statements are included as comments because the users CALL\_CENTER\_USER and REGULATOR\_USER may not exist in your database.
3. This script uses small sample data only. In production telecom systems, these tables may contain millions or billions of records.

```
=====
```

```
*/
```

```
SET SERVEROUTPUT ON;
```

```
SET DEFINE OFF;
```

```
/*=====
```

```
SECTION 1: CLEANUP OLD OBJECTS
```

```
=====*/
```

```
BEGIN
```

```
EXECUTE IMMEDIATE 'DROP MATERIALIZED VIEW mv_daily_cdr_summary';
```

```
EXCEPTION WHEN OTHERS THEN NULL;
```

```
END;
```

```
/
```

```
BEGIN EXECUTE IMMEDIATE 'DROP VIEW vw_customer_360'; EXCEPTION WHEN OTHERS THEN  
NULL; END;
```

```
/
```

```
BEGIN EXECUTE IMMEDIATE 'DROP VIEW vw_regulator_report'; EXCEPTION WHEN OTHERS  
THEN NULL; END;
```

```
/
```

```
BEGIN EXECUTE IMMEDIATE 'DROP VIEW vw_ceo_dashboard'; EXCEPTION WHEN OTHERS THEN  
NULL; END;
```

```
/
```

```
BEGIN EXECUTE IMMEDIATE 'DROP VIEW vw_network_health'; EXCEPTION WHEN OTHERS
THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP VIEW vw_monthly_billing'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP VIEW vw_package_revenue'; EXCEPTION WHEN OTHERS
THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP VIEW vw_daily_recharge'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP VIEW vw_active_subscribers'; EXCEPTION WHEN OTHERS
THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP VIEW vw_customer_service'; EXCEPTION WHEN OTHERS
THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP VIEW vw_finance_revenue'; EXCEPTION WHEN OTHERS
THEN NULL; END;
/

BEGIN EXECUTE IMMEDIATE 'DROP TABLE complaints CASCADE CONSTRAINTS'; EXCEPTION
WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE bills CASCADE CONSTRAINTS'; EXCEPTION WHEN
OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE invoices CASCADE CONSTRAINTS'; EXCEPTION WHEN
OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE network_alarm CASCADE CONSTRAINTS'; EXCEPTION
WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE cdr CASCADE CONSTRAINTS'; EXCEPTION WHEN
OTHERS THEN NULL; END;
/
```

```

BEGIN EXECUTE IMMEDIATE 'DROP TABLE recharges CASCADE CONSTRAINTS'; EXCEPTION
WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE recharge_transactions CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE subscriber_packages CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE subscribers CASCADE CONSTRAINTS'; EXCEPTION
WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE packages CASCADE CONSTRAINTS'; EXCEPTION WHEN
OTHERS THEN NULL; END;
/

```

```

/*=====
SECTION 2: BASE TABLES
=====*/

```

-- Package master table used by CRM and customer 360 use cases.

```

CREATE TABLE packages
(
    package_id    NUMBER PRIMARY KEY,
    package_name  VARCHAR2(50) NOT NULL,
    monthly_fee   NUMBER(12,2) NOT NULL
);

```

-- Main subscriber table.

-- Sensitive columns such as NRC\_NO, PASSWORD, and CARD\_NO should not be exposed  
-- directly to customer service or reporting users.

```

CREATE TABLE subscribers
(
    subscriber_id NUMBER PRIMARY KEY,
    msisdn        VARCHAR2(20) UNIQUE NOT NULL,
    customer_name  VARCHAR2(100) NOT NULL,
    nrc_no         VARCHAR2(50),
    password       VARCHAR2(100),

```

```

card_no    VARCHAR2(30),
address    VARCHAR2(200),
activation_date DATE,
status     VARCHAR2(20),
region     VARCHAR2(50),
package_id NUMBER,
CONSTRAINT fk_subscriber_package
    FOREIGN KEY (package_id) REFERENCES packages(package_id)
);

```

-- Denormalized package subscription table for finance revenue reporting.

```

CREATE TABLE subscriber_packages
(
    sub_pkg_id    NUMBER PRIMARY KEY,
    subscriber_id NUMBER,
    package_name  VARCHAR2(50),
    monthly_fee   NUMBER(12,2),
    start_date    DATE,
    CONSTRAINT fk_sp_subscriber
        FOREIGN KEY (subscriber_id) REFERENCES subscribers(subscriber_id)
);

```

-- Recharge transaction table used by daily dashboard.

```

CREATE TABLE recharge_transactions
(
    txn_id        NUMBER PRIMARY KEY,
    subscriber_id NUMBER,
    msisdn        VARCHAR2(20),
    amount         NUMBER(12,2),
    recharge_date  DATE,
    CONSTRAINT fk_rt_subscriber
        FOREIGN KEY (subscriber_id) REFERENCES subscribers(subscriber_id)
);

```

-- Alias-style recharges table used by Customer 360 view.

```

CREATE TABLE recharges
(
    recharge_id   NUMBER PRIMARY KEY,

```

```
subscriber_id NUMBER,  
amount      NUMBER(12,2),  
recharge_date DATE,  
CONSTRAINT fk_recharges_subscriber  
    FOREIGN KEY (subscriber_id) REFERENCES subscribers(subscriber_id)  
);
```

-- Call Detail Record table.

-- In real telecom systems this can reach billions of records.

```
CREATE TABLE cdr  
(  
    cdr_id      NUMBER PRIMARY KEY,  
    msisdn     VARCHAR2(20),  
    called_number VARCHAR2(20),  
    call_date   DATE,  
    duration_sec NUMBER,  
    charge_amount NUMBER(12,2),  
    call_type   VARCHAR2(20)  
);
```

-- Network alarm table for NOC monitoring.

```
CREATE TABLE network_alarm  
(  
    alarm_id   NUMBER PRIMARY KEY,  
    region     VARCHAR2(50),  
    site_code  VARCHAR2(50),  
    severity   VARCHAR2(20),  
    alarm_date DATE,  
    description VARCHAR2(200)  
);
```

-- Invoice table for read-only finance view.

```
CREATE TABLE invoices  
(  
    invoice_no  VARCHAR2(30) PRIMARY KEY,  
    customer_id NUMBER,  
    amount     NUMBER(12,2),  
    invoice_date DATE,
```

```
    status    VARCHAR2(20)
);
```

-- Bills table for CRM/customer lifecycle information.

```
CREATE TABLE bills
```

```
(
    bill_id    NUMBER PRIMARY KEY,
    subscriber_id NUMBER,
    bill_month  VARCHAR2(7),
    amount     NUMBER(12,2),
    status     VARCHAR2(20),
    CONSTRAINT fk_bills_subscriber
        FOREIGN KEY (subscriber_id) REFERENCES subscribers(subscriber_id)
);
```

-- Complaint table for Customer 360 view.

```
CREATE TABLE complaints
```

```
(
    complaint_id NUMBER PRIMARY KEY,
    subscriber_id NUMBER,
    complaint_date DATE,
    category     VARCHAR2(50),
    status       VARCHAR2(20),
    CONSTRAINT fk_complaints_subscriber
        FOREIGN KEY (subscriber_id) REFERENCES subscribers(subscriber_id)
);
```

```
/*=====
SECTION 3: SAMPLE DATA FOR TESTING
=====*/
```

```
INSERT INTO packages VALUES (1, '5GB Data', 5000);
INSERT INTO packages VALUES (2, 'Unlimited Data', 20000);
INSERT INTO packages VALUES (3, 'Voice Plus', 8000);
INSERT INTO packages VALUES (4, 'Business Premium', 50000);
```

```
INSERT INTO subscribers VALUES
```

```

(1001, '959123456789', 'Mg Mg', '12/ABC(N)123456', 'HASHED_PWD_001',
'411111*****1111', 'Yangon, Kamayut', DATE '2025-01-15', 'ACTIVE', 'Yangon', 1);
INSERT INTO subscribers VALUES
(1002, '959987654321', 'Aye Aye', '9/MYA(N)654321', 'HASHED_PWD_002',
'555555*****4444', 'Mandalay, Chan Aye Tharzan', DATE '2025-02-20', 'ACTIVE', 'Mandalay',
2);
INSERT INTO subscribers VALUES
(1003, '959555111222', 'Ko Ko', '7/BGO(N)112233', 'HASHED_PWD_003', '400000*****9999',
'Bago', DATE '2025-03-10', 'INACTIVE', 'Bago', 3);
INSERT INTO subscribers VALUES
(1004, '959777888999', 'Su Su', '14/PTH(N)778899', 'HASHED_PWD_004', '510000*****8888',
'Ayeyarwady', DATE '2025-04-05', 'ACTIVE', 'Ayeyarwady', 4);
INSERT INTO subscribers VALUES
(1005, '959222333444', 'Hla Hla', '5/SGG(N)223344', 'HASHED_PWD_005',
'422222*****3333', 'Sagaing', DATE '2025-05-01', 'SUSPENDED', 'Sagaing', 1);

INSERT INTO subscriber_packages VALUES (1, 1001, '5GB Data', 5000, DATE '2025-01-15');
INSERT INTO subscriber_packages VALUES (2, 1002, 'Unlimited Data', 20000, DATE '2025-02-
20');
INSERT INTO subscriber_packages VALUES (3, 1003, 'Voice Plus', 8000, DATE '2025-03-10');
INSERT INTO subscriber_packages VALUES (4, 1004, 'Business Premium', 50000, DATE '2025-04-
05');
INSERT INTO subscriber_packages VALUES (5, 1005, '5GB Data', 5000, DATE '2025-05-01');

INSERT INTO recharge_transactions VALUES (1, 1001, '959123456789', 5000, TRUNC(SYSDATE));
INSERT INTO recharge_transactions VALUES (2, 1001, '959123456789', 10000, TRUNC(SYSDATE)
+ 0.25);
INSERT INTO recharge_transactions VALUES (3, 1002, '959987654321', 20000,
TRUNC(SYSDATE));
INSERT INTO recharge_transactions VALUES (4, 1003, '959555111222', 3000, TRUNC(SYSDATE) -
1);
INSERT INTO recharge_transactions VALUES (5, 1004, '959777888999', 50000,
TRUNC(SYSDATE));
INSERT INTO recharge_transactions VALUES (6, 1005, '959222333444', 5000, TRUNC(SYSDATE) -
2);

INSERT INTO recharges VALUES (1, 1001, 5000, TRUNC(SYSDATE));
INSERT INTO recharges VALUES (2, 1001, 10000, TRUNC(SYSDATE));

```

```
INSERT INTO recharges VALUES (3, 1002, 20000, TRUNC(SYSDATE));
INSERT INTO recharges VALUES (4, 1003, 3000, TRUNC(SYSDATE) - 1);
INSERT INTO recharges VALUES (5, 1004, 50000, TRUNC(SYSDATE));
INSERT INTO recharges VALUES (6, 1005, 5000, TRUNC(SYSDATE) - 2);
```

```
INSERT INTO cdr VALUES (1, '959123456789', '959111111111', DATE '2026-06-01' + 10/24, 120,
200, 'VOICE');
INSERT INTO cdr VALUES (2, '959123456789', '959222222222', DATE '2026-06-01' + 11/24, 300,
500, 'VOICE');
INSERT INTO cdr VALUES (3, '959123456789', '959333333333', DATE '2026-06-02' + 9/24, 60,
100, 'VOICE');
INSERT INTO cdr VALUES (4, '959987654321', '959444444444', DATE '2026-06-01' + 8/24, 600,
1000, 'VOICE');
INSERT INTO cdr VALUES (5, '959987654321', '959555555555', DATE '2026-06-02' + 13/24, 180,
300, 'VOICE');
INSERT INTO cdr VALUES (6, '959777888999', '959666666666', DATE '2026-06-02' + 14/24, 900,
1500, 'VOICE');
```

```
INSERT INTO network_alarm VALUES (1, 'Yangon', 'YGN001', 'CRITICAL', TRUNC(SYSDATE), 'Core
router down');
INSERT INTO network_alarm VALUES (2, 'Yangon', 'YGN002', 'MAJOR', TRUNC(SYSDATE), 'High
packet loss');
INSERT INTO network_alarm VALUES (3, 'Mandalay', 'MDY001', 'CRITICAL', TRUNC(SYSDATE),
'Base station offline');
INSERT INTO network_alarm VALUES (4, 'Mandalay', 'MDY002', 'MINOR', TRUNC(SYSDATE),
'Power backup warning');
INSERT INTO network_alarm VALUES (5, 'Bago', 'BGO001', 'CRITICAL', TRUNC(SYSDATE) - 1,
'Fiber cut yesterday');
```

```
INSERT INTO invoices VALUES ('INV-2026-0001', 1001, 15000, DATE '2026-06-01', 'PAID');
INSERT INTO invoices VALUES ('INV-2026-0002', 1002, 20000, DATE '2026-06-01', 'UNPAID');
INSERT INTO invoices VALUES ('INV-2026-0003', 1004, 50000, DATE '2026-06-01', 'PAID');
```

```
INSERT INTO bills VALUES (1, 1001, '2026-06', 15000, 'PAID');
INSERT INTO bills VALUES (2, 1002, '2026-06', 20000, 'UNPAID');
INSERT INTO bills VALUES (3, 1004, '2026-06', 50000, 'PAID');
```

```
INSERT INTO complaints VALUES (1, 1001, TRUNC(SYSDATE), 'Network', 'OPEN');
```



```
INSERT INTO complaints VALUES (2, 1001, TRUNC(SYSDATE) - 1, 'Billing', 'CLOSED');
INSERT INTO complaints VALUES (3, 1002, TRUNC(SYSDATE), 'Recharge', 'OPEN');
INSERT INTO complaints VALUES (4, 1004, TRUNC(SYSDATE), 'Enterprise Support',
'IN_PROGRESS');
```

```
COMMIT;
```

```
/*=====
SECTION 4: SCENARIO 1 - CUSTOMER SERVICE VIEW
Purpose: Hide NRC, password, and card information from call center users.
=====*/
```

```
CREATE OR REPLACE VIEW vw_customer_service
AS
SELECT
    subscriber_id,
    msisdn,
    customer_name,
    address,
    activation_date,
    status,
    region
FROM subscribers;
```

```
SQL> SELECT * FROM vw_customer_service;
```

```
SUBSCRIBER_ID MSISDN
```

```
CUSTOMER_NAME
```

```
ADDRESS
```

```
ACTIVATIO STATUS
```

```
REGION
```

```
1001 959123456789
```

```
Mg Mg
```

```
Yangon, Kamayut
```

```
SUBSCRIBER_ID MSISDN
```

```
CUSTOMER_NAME
```

```
ADDRESS
```

```
ACTIVATIO STATUS
```

```
REGION
```

```
15-JAN-25 ACTIVE
```

```
Yangon
```

```
-- Security grant example:
```

```
-- GRANT SELECT ON vw_customer_service TO call_center_user;
```

```
/*=====
SECTION 5: SCENARIO 2 - ACTIVE SUBSCRIBERS VIEW
Purpose: Marketing team sees only active subscribers.
=====*/
```

```
CREATE OR REPLACE VIEW vw_active_subscribers
```

```
AS
```

```
SELECT
```

```
    s.subscriber_id,
```

```
    s.msisdn,
```

```
    s.customer_name,
```

```
    p.package_name,
```

```
    s.region,
```

```
    s.activation_date
```

```
FROM subscribers s
```

```

JOIN packages p
  ON s.package_id = p.package_id
WHERE s.status = 'ACTIVE';

```

```

SQL> SELECT * FROM vw_active_subscribers;

SUBSCRIBER_ID MSISDN
-----
CUSTOMER_NAME
-----
PACKAGE_NAME
-----
REGION ACTIVATIO
-----
          1001 959123456789
Mg Mg
5GB Data
Yangon 15-JAN-25

SUBSCRIBER_ID MSISDN
-----
CUSTOMER_NAME
-----
PACKAGE_NAME
-----
REGION ACTIVATIO
-----
          1002 959987654321
Aye Aye
Unlimited Data
Mandalay 20-FEB-25

```

```

/*=====
SECTION 6: SCENARIO 3 - DAILY RECHARGE DASHBOARD
Purpose: Daily recharge count and revenue dashboard.
=====*/

```

```

CREATE OR REPLACE VIEW vw_daily_recharge
AS
SELECT
  TRUNC(recharge_date) AS recharge_day,
  COUNT(*) AS total_recharges,
  SUM(amount) AS total_amount
FROM recharge_transactions
GROUP BY TRUNC(recharge_date);

```

```
SQL> SELECT * FROM vw_daily_recharge;
```

| RECHARGE_ | TOTAL_RECHARGES | TOTAL_AMOUNT |
|-----------|-----------------|--------------|
| 08-JUN-26 | 4               | 85000        |
| 07-JUN-26 | 1               | 3000         |
| 06-JUN-26 | 1               | 5000         |

```
/*=====
SECTION 7: SCENARIO 4 - REVENUE BY PRODUCT VIEW
Purpose: Finance department monitors revenue by package/product.
=====*/
```

```
CREATE OR REPLACE VIEW vw_package_revenue
```

```
AS
```

```
SELECT
```

```
    package_name,
```

```
    COUNT(*) AS subscribers,
```

```
    SUM(monthly_fee) AS revenue
```

```
FROM subscriber_packages
```

```
GROUP BY package_name;
```

```
SQL> SELECT * FROM vw_package_revenue;
```

| PACKAGE_NAME     | SUBSCRIBERS | REVENUE |
|------------------|-------------|---------|
| 5GB Data         | 2           | 10000   |
| Unlimited Data   | 1           | 20000   |
| Voice Plus       | 1           | 8000    |
| Business Premium | 1           | 50000   |

```
/*=====
SECTION 8: SCENARIO 5 - TELECOM MONTHLY BILLING VIEW
Purpose: Monthly call usage and charge per MSISDN.
=====*/
```

```
CREATE OR REPLACE VIEW vw_monthly_billing
```

```
AS
```

```
SELECT
```

```
    msisdn,
```

```
    TO_CHAR(call_date, 'YYYY-MM') AS billing_month,
```

```
    SUM(duration_sec) AS total_seconds,
```

```

SUM(charge_amount) AS total_charge
FROM cdr
GROUP BY
    msisdn,
    TO_CHAR(call_date, 'YYYY-MM');

```

```
SQL> SELECT * FROM vw_monthly_billing;
```

| MSISDN       | BILLING | TOTAL_SECONDS | TOTAL_CHARGE |
|--------------|---------|---------------|--------------|
| 959123456789 | 2026-06 | 480           | 800          |
| 959987654321 | 2026-06 | 780           | 1300         |
| 959777888999 | 2026-06 | 900           | 1500         |

```

/*=====
SECTION 9: SCENARIO 6 - NETWORK OPERATIONS DASHBOARD
Purpose: NOC team monitors today network failures by region.
=====*/

```

```

CREATE OR REPLACE VIEW vw_network_health
AS
SELECT
    region,
    COUNT(*) AS total_alarms,
    SUM(CASE WHEN severity = 'CRITICAL' THEN 1 ELSE 0 END) AS critical_count,
    SUM(CASE WHEN severity = 'MAJOR' THEN 1 ELSE 0 END) AS major_count,
    SUM(CASE WHEN severity = 'MINOR' THEN 1 ELSE 0 END) AS minor_count
FROM network_alarm
WHERE alarm_date >= TRUNC(SYSDATE)
GROUP BY region;

```

```
SQL> SELECT * FROM vw_network_health;
```

| REGION      | TOTAL_ALARMS |   | CRITICAL_COUNT |
|-------------|--------------|---|----------------|
| MAJOR_COUNT | MINOR_COUNT  |   |                |
| Yangon      | 1            | 0 | 2              |
| Mandalay    | 0            | 1 | 2              |

```

/*=====
SECTION 10: SCENARIO 7 - EXECUTIVE KPI VIEW
Purpose: CEO dashboard with key telecom KPIs.
=====*/

```

```

CREATE OR REPLACE VIEW vw_ceo_dashboard
AS
SELECT
    (SELECT COUNT(*)
     FROM subscribers
     WHERE status = 'ACTIVE') AS active_subscribers,
    (SELECT NVL(SUM(amount), 0)
     FROM recharge_transactions
     WHERE recharge_date >= TRUNC(SYSDATE)) AS daily_revenue,
    (SELECT COUNT(*)
     FROM network_alarm
     WHERE severity = 'CRITICAL'
     AND alarm_date >= TRUNC(SYSDATE)) AS today_critical_alarms,
    (SELECT NVL(SUM(charge_amount), 0)
     FROM cdr
     WHERE call_date >= DATE '2026-06-01'
     AND call_date < DATE '2026-07-01') AS june_2026_cdr_revenue
FROM dual;

```

```

SQL> SELECT * FROM vw_ceo_dashboard;

```

| ACTIVE_SUBSCRIBERS | DAILY_REVENUE | TODAY_CRITICAL_ALARMS | JUNE_2026_CDR_REVENUE |
|--------------------|---------------|-----------------------|-----------------------|
| 3                  | 85000         | 2                     | 3600                  |

```

/*=====
SECTION 11: SCENARIO 8 - REGULATORY REPORTING VIEW
Purpose: Regulator sees subscriber counts by region, not individual data.
=====*/

```

```

CREATE OR REPLACE VIEW vw_regulator_report
AS
SELECT
    region,
    status,

```

```

COUNT(*) AS subscriber_count
FROM subscribers
GROUP BY region, status;

```

-- Regulator grant example:

-- GRANT SELECT ON vw\_regulator\_report TO regulator\_user;

```
SQL> SELECT * FROM vw_regulator_report;
```

| REGION     |   | STATUS    |
|------------|---|-----------|
| Yangon     | 1 | ACTIVE    |
| Mandalay   | 1 | ACTIVE    |
| Bago       | 1 | INACTIVE  |
| Ayeyarwady | 1 | ACTIVE    |
| Sagaing    | 1 | SUSPENDED |

```

/*=====
SECTION 12: SCENARIO 9 - READ-ONLY FINANCE VIEW
Purpose: Prevent accidental DML through finance reporting view.
=====*/

```

```

CREATE OR REPLACE VIEW vw_finance_revenue
AS
SELECT
    invoice_no,
    customer_id,
    amount,
    invoice_date,
    status

```

FROM invoices  
WITH READ ONLY;

```
SQL> SELECT * FROM vw_finance_revenue;
```

| INVOICE_NO              | CUSTOMER_ID | AMOUNT | INVOICE_D |
|-------------------------|-------------|--------|-----------|
| INV-2026-0001<br>PAID   | 1001        | 15000  | 01-JUN-26 |
| INV-2026-0002<br>UNPAID | 1002        | 20000  | 01-JUN-26 |
| INV-2026-0003<br>PAID   | 1004        | 50000  | 01-JUN-26 |

/\*=====

### SECTION 13: SCENARIO 10 - MATERIALIZED VIEW FOR CDR SUMMARY

Purpose: Pre-aggregate CDR data for faster reporting.

Training note:

FAST refresh needs materialized view logs and additional restrictions.

For a simple classroom/demo script, REFRESH COMPLETE ON DEMAND is safer.

=====\*/

```
CREATE MATERIALIZED VIEW mv_daily_cdr_summary
BUILD IMMEDIATE
REFRESH COMPLETE
ON DEMAND
AS
SELECT
    TRUNC(call_date) AS call_day,
    COUNT(*) AS total_calls,
    SUM(duration_sec) AS total_duration,
    SUM(charge_amount) AS total_revenue
FROM cdr
GROUP BY TRUNC(call_date);
```



```
SQL> SELECT * FROM mv_daily_cdr_summary;
```

| CALL_DAY  | TOTAL_CALLS | TOTAL_DURATION | TOTAL_REVENUE |
|-----------|-------------|----------------|---------------|
| 01-JUN-26 | 3           | 1020           | 1700          |
| 02-JUN-26 | 3           | 1140           | 1900          |

-- Manual refresh example:

-- EXEC DBMS\_MVIEW.REFRESH('MV\_DAILY\_CDR\_SUMMARY');

```
/*=====
SECTION 14: SCENARIO 11 - CUSTOMER 360 VIEW
Purpose: CRM application gets a single customer profile from multiple tables.
=====*/
```

CREATE OR REPLACE VIEW vw\_customer\_360

AS

SELECT

s.subscriber\_id,  
 s.msisdn,  
 s.customer\_name,  
 s.status,  
 s.region,  
 p.package\_name,  
 NVL(SUM(DISTINCT r.amount), 0) AS total\_recharge,  
 COUNT(DISTINCT c.complaint\_id) AS complaints,  
 NVL(SUM(DISTINCT b.amount), 0) AS total\_billed\_amount

FROM subscribers s

LEFT JOIN packages p

ON s.package\_id = p.package\_id

LEFT JOIN recharges r

ON s.subscriber\_id = r.subscriber\_id

LEFT JOIN complaints c

ON s.subscriber\_id = c.subscriber\_id

LEFT JOIN bills b

ON s.subscriber\_id = b.subscriber\_id

GROUP BY

s.subscriber\_id,  
 s.msisdn,

s.customer\_name,  
s.status,  
s.region,  
p.package\_name;

```
SQL> SELECT * FROM vw_customer_360;

SUBSCRIBER_ID MSISDN
-----
CUSTOMER_NAME
-----
STATUS          REGION
-----
PACKAGE_NAME          TOTAL_RECHARGE COMPLAINTS
-----
TOTAL_BILLED_AMOUNT
-----
1001 959123456789
Mg Mg
ACTIVE          Yangon

SUBSCRIBER_ID MSISDN
-----
CUSTOMER_NAME
-----
STATUS          REGION
-----
PACKAGE_NAME          TOTAL_RECHARGE COMPLAINTS
-----
TOTAL_BILLED_AMOUNT
-----
5GB Data          15000          2
15000
```

```
/*=====
SECTION 15: TESTING QUERIES
Run these SELECT statements one by one in SQL Developer.
=====*/
```

```
SQL> PROMPT Scenario 1: Customer Service View - sensitive columns hidden
SELECT * FROM vw_customer_service ORDER BY subscriber_id;
Scenario 1: Customer Service View - sensitive columns hidden
SQL>
SUBSCRIBER_ID MSISDN
-----
CUSTOMER_NAME
-----
ADDRESS
-----
ACTIVATIO STATUS
-----
REGION
-----
          1001 959123456789
Mg Mg
Yangon, Kamayut

SUBSCRIBER_ID MSISDN
-----
CUSTOMER_NAME
-----
ADDRESS
-----
ACTIVATIO STATUS
-----
REGION
-----
15-JAN-25 ACTIVE
Yangon
```

```
SQL> PROMPT Scenario 2: Active Subscribers View
SELECT * FROM vw_active_subscribers ORDER BY subscriber_id;
Scenario 2: Active Subscribers View
SQL>
SUBSCRIBER_ID MSISDN
-----
CUSTOMER_NAME
-----
PACKAGE_NAME
-----
REGION ACTIVATIO
-----
1001 959123456789
Mg Mg
5GB Data
Yangon 15-JAN-25
```

```
SUBSCRIBER_ID MSISDN
-----
CUSTOMER_NAME
-----
PACKAGE_NAME
-----
REGION ACTIVATIO
-----
1002 959987654321
Aye Aye
Unlimited Data
Mandalay 20-FEB-25
```

```
SQL> PROMPT Scenario 2: Active subscriber count for marketing
SELECT COUNT(*) AS active_subscriber_count FROM vw_active_subscribers;
Scenario 2: Active subscriber count for marketing
SQL>
ACTIVE_SUBSCRIBER_COUNT
-----
3
```

```
SQL> PROMPT Scenario 3: Daily Recharge Dashboard
SELECT * FROM vw_daily_recharge ORDER BY recharge_day DESC;
Scenario 3: Daily Recharge Dashboard
SQL>
RECHARGE_ TOTAL_RECHARGES TOTAL_AMOUNT
-----
08-JUN-26 4 85000
07-JUN-26 1 3000
06-JUN-26 1 5000
```

```
SQL> PROMPT Scenario 4: Package Revenue
SELECT * FROM vw_package_revenue ORDER BY revenue DESC;
Scenario 4: Package Revenue
SQL>
```

| PACKAGE_NAME     | SUBSCRIBERS | REVENUE |
|------------------|-------------|---------|
| Business Premium | 1           | 50000   |
| Unlimited Data   | 1           | 20000   |
| 5GB Data         | 2           | 10000   |
| Voice Plus       | 1           | 8000    |

```
SQL> PROMPT Scenario 5: Monthly Billing for one MSISDN
SELECT *
FROM vw_monthly_billing
WHERE msisdn = '959123456789';
Scenario 5: Monthly Billing for one MSISDN
SQL> 2 3
```

| MSISDN       | BILLING | TOTAL_SECONDS | TOTAL_CHARGE |
|--------------|---------|---------------|--------------|
| 959123456789 | 2026-06 | 480           | 800          |

```
SQL> PROMPT Scenario 6: Network Health Today
SELECT * FROM vw_network_health ORDER BY critical_count DESC;
Scenario 6: Network Health Today
SQL>
```

| REGION   | TOTAL_ALARMS | CRITICAL_COUNT |
|----------|--------------|----------------|
| Mandalay | 2            | 1              |
| Yangon   | 2            | 1              |

```
SQL> PROMPT Scenario 7: CEO Dashboard
SELECT * FROM vw_ceo_dashboard;
Scenario 7: CEO Dashboard
SQL>
```

| ACTIVE_SUBSCRIBERS | DAILY_REVENUE | TODAY_CRITICAL_ALARMS | JUNE_2026_CDR_REVENUE |
|--------------------|---------------|-----------------------|-----------------------|
| 3                  | 85000         | 2                     | 3600                  |

```
SQL> PROMPT Scenario 8: Regulator Report
SELECT * FROM vw_regulator_report ORDER BY region, status;
Scenario 8: Regulator Report
SQL>
```

| REGION           |   | STATUS    |
|------------------|---|-----------|
| -----            |   |           |
| SUBSCRIBER_COUNT |   |           |
| -----            |   |           |
| Ayeyarwady       | 1 | ACTIVE    |
| Bago             | 1 | INACTIVE  |
| Mandalay         | 1 | ACTIVE    |
| -----            |   |           |
| REGION           |   | STATUS    |
| -----            |   |           |
| SUBSCRIBER_COUNT |   |           |
| -----            |   |           |
| Sagaing          | 1 | SUSPENDED |
| Yangon           | 1 | ACTIVE    |

```
SQL> PROMPT Scenario 9: Read-Only Finance View
SELECT * FROM vw_finance_revenue ORDER BY invoice_no;
Scenario 9: Read-Only Finance View
SQL>
```

| INVOICE_NO              | CUSTOMER_ID | AMOUNT | INVOICE_D |
|-------------------------|-------------|--------|-----------|
| -----                   |             |        |           |
| STATUS                  |             |        |           |
| -----                   |             |        |           |
| INV-2026-0001<br>PAID   | 1001        | 15000  | 01-JUN-26 |
| INV-2026-0002<br>UNPAID | 1002        | 20000  | 01-JUN-26 |
| INV-2026-0003<br>PAID   | 1004        | 50000  | 01-JUN-26 |

```
SQL> PROMPT Scenario 9 Test: This UPDATE should fail with a read-only view error
UPDATE vw_finance_revenue SET amount = 0 WHERE invoice_no = 'INV-2026-0001';
Scenario 9 Test: This UPDATE should fail with a read-only view error
SQL> UPDATE vw_finance_revenue SET amount = 0 WHERE invoice_no = 'INV-2026-0001'
*
```

```
ERROR at line 1:
ORA-42399: cannot perform a DML operation on a read-only view
```

```
SQL> PROMPT Scenario 10: Materialized View Daily CDR Summary
SELECT * FROM mv_daily_cdr_summary ORDER BY call_day;
Scenario 10: Materialized View Daily CDR Summary
SQL>
```

| CALL_DAY  | TOTAL_CALLS | TOTAL_DURATION | TOTAL_REVENUE |
|-----------|-------------|----------------|---------------|
| 01-JUN-26 | 3           | 1020           | 1700          |
| 02-JUN-26 | 3           | 1140           | 1900          |

```
SQL> PROMPT Scenario 11: Customer 360 View
```

```
SELECT *
FROM vw_customer_360
WHERE msisdn = '959123456789';
Scenario 11: Customer 360 View
SQL> 2 3
```

```
SUBSCRIBER_ID MSISDN
```

```
CUSTOMER_NAME
```

```
STATUS REGION
```

```
PACKAGE_NAME
```

```
TOTAL_RECHARGE COMPLAINTS
```

```
TOTAL_BILLED_AMOUNT
```

```
1001 959123456789
```

```
Mg Mg
```

```
ACTIVE Yangon
```

```
SUBSCRIBER_ID MSISDN
```

```
CUSTOMER_NAME
```

```
STATUS REGION
```

```
PACKAGE_NAME
```

```
TOTAL_RECHARGE COMPLAINTS
```

```
TOTAL_BILLED_AMOUNT
```

```
5GB Data
```

```
15000
```

```
15000
```

```
2
```

```
/*=====
SECTION 16: RECOMMENDED INDEXES FOR PROFESSIONAL TELECOM SYSTEMS
These are optional but useful for performance in larger datasets.
=====*/
```

```
CREATE INDEX idx_subscribers_status ON subscribers(status);
CREATE INDEX idx_subscribers_region ON subscribers(region);
CREATE INDEX idx_recharge_date ON recharge_transactions(recharge_date);
CREATE INDEX idx_cdr_msisdn_month ON cdr(msisdn, call_date);
CREATE INDEX idx_network_alarm_date ON network_alarm(alarm_date, severity);
```

```
/*=====
END OF SCRIPT
=====*/
```